SYSTEM DESIGN INTERVIEW

Design Facebook News Feed

personalised, ML-ranked timeline of friends, pages, and groups at planet scale

~ 45 minute read

What's Inside

01 Problem Statement

Requirements and scope

Design **Facebook News Feed**: the home screen that shows each user a personalised stream of posts from **friends, pages, and groups** they follow. The feed is **not chronological** — it is ML-ranked. The system must ingest ~1B posts/day, serve ~10B feed reads/day, rank with ≤300 ms latency, and remain available globally.

Key Features

- Personalised home feed — posts from friends, followed pages, joined groups
- ML ranking by engagement, affinity, recency, content type, negative signals
- Pull-to-refresh + infinite scroll; pagination with stable cursors
- Mixed content types — text, photo, video, link preview, life events, ads
- Engagement primitives — react, comment, share, save, hide, snooze, unfollow
- Real-time injection of viral / trending posts
- Cross-device sync — last seen position, dismissed posts, see-first lists

Clarifying Questions

Question	Answer
Scope?	Home News Feed only; profile timelines, Stories, Reels out of scope
Scale?	3B MAU, 2B DAU, ~1B posts/day, ~10B feed reads/day
Ranking?	ML personalisation; not chronological; explainable signals
Latency?	p99 feed load < 1.5s end-to-end; ranking < 300 ms
Freshness?	< 60 sec for friend posts; viral injection < 5 min
Average friends/follows?	~350 friends + ~200 pages/groups = ~550 sources
Average post candidates per read?	~1,500 fresh + ~3,000 unseen = ~5K to rank
Geography?	Global; multi-region active-active reads, single-region writes per user

KEY

Key Assumption

> Design for **2B DAU**, **1B posts/day**, **10B feed reads/day**, and **ML ranking under 300 ms**. Every architectural choice flows from those four numbers.

02 Requirements

Functional and non-functional

Functional Requirements

- **Post:** create text/photo/video/link post; choose audience
- **Feed read:** GET ranked, personalised feed with cursor pagination
- **Engagement:** react, comment, share, save, hide, snooze
- **Negative feedback:** hide, snooze 30d, unfollow, report
- **See-first / favorites:** users pin sources for promotion
- **Friend / page / group graph:** source of candidate posts
- **Counter sync:** reaction, comment, share counters across devices

Non-Functional Requirements

Requirement	Target
Availability	99.99% for read path; 99.9% for write path
Latency	p50 feed < 400 ms, p99 < 1.5s; ranking < 300 ms
Freshness	Friend posts visible < 60 s; viral injection < 5 min
Consistency	Eventual for feed; read-your-writes for the poster's own post
Throughput	~115K reads/sec avg, ~300K peak; ~12K post writes/sec
Durability	Posts: 11 nines on BLOB / 5 nines on metadata
Scale	10× growth without redesign; horizontal everywhere
Cost	Optimise ranking compute and Memcached fleet — the dominant lines

INSIGHT

Read-Your-Writes

When you publish a post you must see it instantly in your own feed even before fan-out completes. We solve this with a **self-injection** step: the poster's own client receives a synthetic feed entry referencing the fresh post_id. Other followers see it after fan-out (push) or on next pull (celebrities).

03 Capacity Estimation

Math for scale

Traffic

- MAU: **3B**; DAU: **2B**
- Posts/day: **1B** → 1B / 86,400 ≈ **11,574 ≈ 12K writes/sec**
- Feed opens/user/day: **5** avg → 2B × 5 = **10B feed reads/day**
- Read QPS avg: 10B / 86,400 ≈ **115,740 ≈ 115K reads/sec**
- Peak QPS (2.5× diurnal skew): **~300K reads/sec**
- Engagement events (react+comment+share): ~10× post rate ≈ **120K/sec**

Ranking Cost

- Candidates ranked per feed read: **~5,000** (after candidate filtering)
- Total feature evaluations/sec: 115K reads × 5K candidates = **575M scores/sec**
- Per-score budget: 300 ms / 5K = **60 µs per candidate**
- → pre-compute embeddings; serve via DLRM inference on GPU/TPU pools
- Top-K returned per read: **~30 stories** (page 1) + paginate

Storage

- Post metadata: ~1 KB × 1B = **1 TB/day** → ~365 TB/year metadata
- Photo BLOB: ~250 KB × 30% of posts (300M) = **75 TB/day** media
- Video BLOB: ~3 MB × 5% of posts (50M) = **150 TB/day** media
- 5-year corpus: ~411 PB media + ~1.8 PB metadata; tiered to cold storage
- Feed timeline (push fan-out): see §05 — ~30B inserts/day, denormalised

Cache & Bandwidth

- Active feed working set: ~200M users opening/hour × ~30 stories × 1 KB ≈ **6 TB hot** Memcached
- Photo egress (CDN): ~10B reads × ~100 KB rendered = **~1 EB/day** at the edge
- Origin egress after CDN: assume 95% offload → **~50 PB/day** from origin
- Inbound media: ~225 TB/day ≈ **~21 Gbps avg, ~60 Gbps peak**

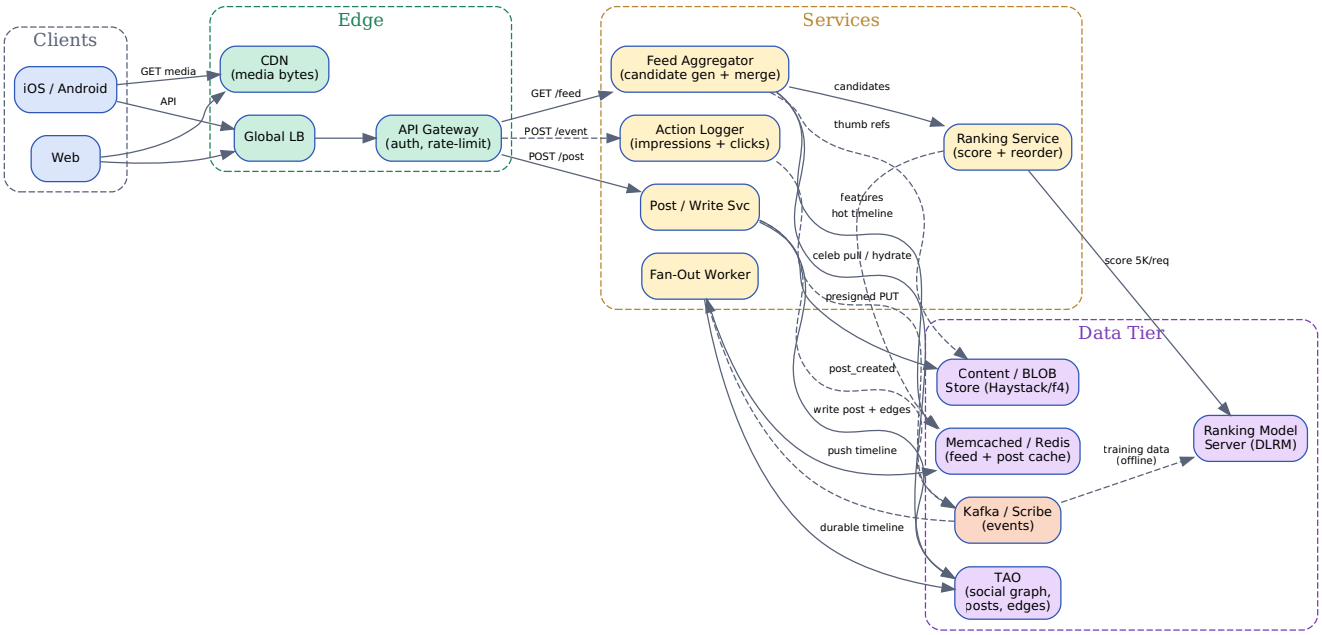
> **KEY**
>
> **Numbers To Memorise**

3B MAU / 2B DAU · **1B posts/day** (12K/sec) · **10B feed reads/day** (115K avg, 300K peak) · **5K candidates/read, 60 µs/score** · **~30 stories** served per page · **~6 TB hot** Memcached working set.

04 High-Level Architecture

System overview

The architecture has three planes. **Ingest** accepts new posts and runs fan-out. **Serve** reads candidates, ranks them, and returns the page. **Logging** captures every impression and engagement so the next training run can improve relevance. Each plane scales independently; Kafka is the only contract between them.



End-to-end: client to API gateway to Feed Aggregator + Ranking Service + Action Logger; data tier mixes TAO graph, BLOB store, Memcached, the Ranking Model Server, and Kafka.

Service Responsibilities

- **API Gateway:** authn, per-user QPS limits, request routing
- **Post Service:** writes post row to TAO, issues presigned BLOB URL, emits `post_created`
- **Fan-Out Worker:** Kafka consumer; pushes post_id into followers' Memcached timelines (regular users) or skips for celebrities
- **Feed Aggregator:** on read, fetches pre-built timelines + celebrity candidates, merges, dedups, hydrates
- **Ranking Service:** calls the Model Server with feature vectors; returns top-K
- **Action Logger:** async log of every impression, dwell, click, react, hide — feeds offline training
- **Ranking Model Server:** hosts the trained DLRM; ~60 µs/inference on accelerator
- **TAO:** graph store of users, posts, edges (friend/follows/likes); cache-fronted MySQL
- **BLOB store:** Haystack (warm) + f4 (cold) for photos and videos
- **Memcached:** per-user feed lists, post metadata, ranking features

- **Kafka / Scribe:** backbone for events and async work

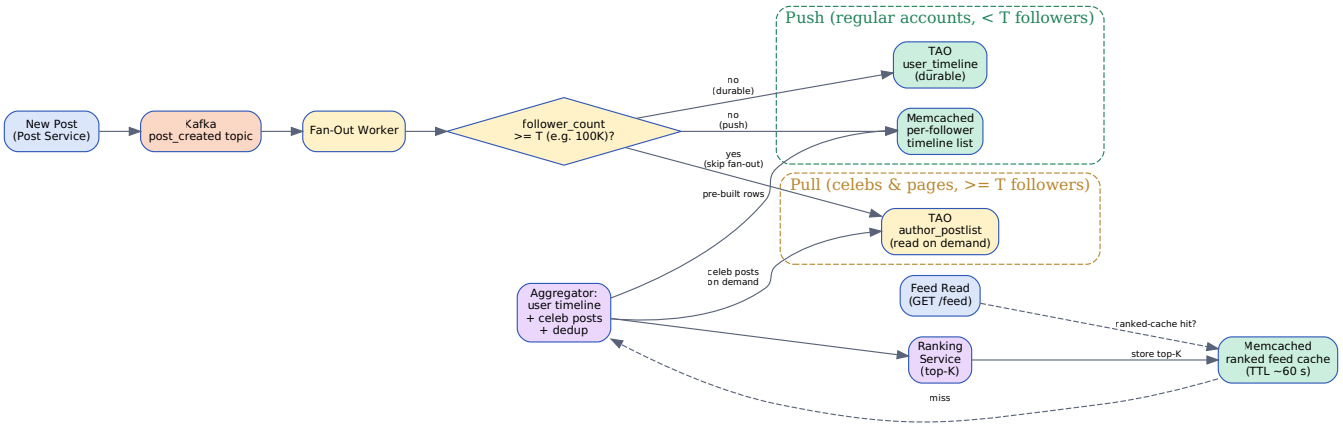
05 Hybrid Push / Pull Fan-Out

The core feed-delivery decision

Naïve approaches break fast. **Pure push** melts when a page with 100M followers posts. **Pure pull** melts when 2B DAU each scatter-gather across 550 sources at read time. Facebook uses a **hybrid**: push for normal accounts, pull for high-fanout sources, merge on read.

Push vs. Pull vs. Hybrid

Aspect	Push (write fan-out)	Pull (read fan-out)	Hybrid (recommended)
Read latency	Fast — timeline pre-built	Slow — scatter-gather across 550 sources	Fast for both paths
Write cost	O(followers) per post	O(1) per post	O(followers) below threshold; O(1) above
Storage	N timeline entries per post	1 row in postDB	Mostly N copies; 0 for celebs
Worst case	Page with 100M followers → 100M writes	Active user open → 550-way fan-in	Celeb post: O(1); user open: ~20-way fan-in
Freshness	Eventually consistent (push lag)	Always fresh on read	Push lag < 60 s; pull always fresh
Best for	Low-fanout users	Very-high-fanout sources	Realistic distribution (heavy-tail)



Hybrid fan-out: regular users push into followers' Memcached timelines + TAO; celebrities & big pages skip the push and live in TAO postDB; the read path merges both at request time.

Fan-Out Worker — Pseudocode

```
def on_post_created(event):
    post   = event.post
    author = tao.get_user(post.user_id)
    fc     = author.follower_count

    # Always durable-write the post once on the author side
    tao.append_author_postlist(author.id, post.id, post.created_at)

    if fc >= PUSH_THRESHOLD:                        # celeb / big page
        metrics.incr('fanout.skip_celeb')
        return                                       # readers will pull

    # Push path — page through followers in batches
    for batch in tao.followers_paged(author.id, 5000):
        ops = []
        for follower_id in batch:
            if user_pref.is_snoozed(follower_id, author.id):
                continue                             # skip muted
            key = f'tl:{follower_id}'
            ops.append(memcache.lpush(key,
                       (post.id, post.created_at, author.id),
                       maxlen=2000))                 # cap timeline size
        memcache.pipeline(ops)                        # batch RPC
        tao.append_user_timeline_batch(batch, post)    # durable copy

    metrics.incr('fanout.push_done', fc)
```

Threshold-Tuning Math (push vs. pull break-even)

KEY

Where Does The Push/Pull Curve Cross?

Let **F** = an author's follower count, **R** = average reads per follower per day (assume 5), **w_push** = cost of one Memcached LPUSH (~80 µs incl. network), **w_pull** = cost of one extra TAO partition read on the read path (~600 µs incl. cache miss assumption).

Push cost per post: `F · w_push` = F × 80 µs.
Pull cost per post: incurred on every read by every follower = `F · R · w_pull` = F × 5 × 600 µs *per day*.

Per-post push is paid once; pull is paid R times per follower per day for the post's lifetime in feed (call it L = 2 days of decay). Equating cost per post:
`F · w_push ≈ F · R · L · w_pull` simplifies away — the per-post cost is linear in F either way. **So the break-even isn't on F directly — it's on the *read multiplier* R·L vs the push multiplier 1.** Push wins

> when R·L · w_pull > w_push, i.e. 5 × 2 × 600 µs = 6,000 µs > 80 µs — which is always true.
>
> **So why ever pull?** Because push has a *burst* cost: F LPUSHs in seconds. A page with 100M followers triggers 100M LPUSH at one instant — saturating the cluster. Pull amortises across the day. The real threshold isn't where averages cross — it's where **burst push exceeds your cluster's instantaneous capacity**: `T = capacity_per_sec · max_seconds_to_complete / posts_per_sec_from_class`. For Facebook's measured cluster, **T ≈ 100K followers** is the operational sweet spot.

Hybrid Algorithm — Read Path

1. Look up user's pre-built timeline list in Memcached (push path output)
2. Look up the user's **celeb-follow set** (cached); for each, query TAO `author_postlist` for the last N posts
3. Merge by created_at; **dedup** by post_id; cap to ~5K candidates
4. Hydrate metadata (author name, counters, BLOB refs) from Memcached + TAO
5. Send candidates + user features to Ranking Service
6. Persist ranked top-K in Memcached for ~60 s (handles re-pulls during scroll)

06 ML Ranking — Signals & Models

EdgeRank → DLRM

Chronological ordering buries good content. The ranker assigns each candidate a score and returns top-K. The model has evolved: original **EdgeRank** was a 3-term linear formula; today's production is a **DLRM** (Deep Learning Recommendation Model) with hundreds of features and learned embeddings, served on accelerators.

Ranking Signals

Signal	What it captures
Affinity (u_e)	How often viewer u interacts with author/source e (likes, comments, profile visits, DMs)
Edge weight (w_e)	Importance of the edge type — comments > reactions > views; videos > photos > text
Time decay (d_e)	Exponential decay; freshness boost; half-life ~6 hours for friends, ~3 hours for pages
Negative signals	Hide, snooze, unfollow, low-dwell, report — strong demotions
Content quality	Click-bait classifier, integrity filters, NSFW, misinformation flags
Diversity	Anti-echo: cap stories from one author/page in top-K
Predicted action	P(react), P(comment), P(share), P(dwell > 3s) — multi-task heads
Personal preferences	See-first list, snooze list, language, locality

EdgeRank (v0)

```
# Original EdgeRank — sum over edges (u, e_i) for candidate post p
score(u, p) = sum_over_edges(
    affinity(u, e) * weight(e) * decay(e)
)

affinity(u, e)  = log(1 + interaction_count_30d(u, author(e))) / 5
weight(e)       = {'comment': 3.0, 'react': 1.0,
                   'share':   2.5, 'view':  0.2}[e.type]
decay(e)        = exp(-ln(2) * age_hours(e) / half_life_hours)
negative_pen(u, p) = -10.0 if hidden_similar(u, p) else 0

# Sort candidates by score desc; return top-K
```

DLRM (production)

- **Inputs:** dense user features + dense item features + 100s of categorical IDs (author, page, hashtag, language, device, hour-of-day…)
- **Embedding tables:** 10s of TB of learned vectors; sharded across parameter servers
- **Bottom MLP:** processes dense features; **top MLP:** combines with embedding interactions (dot products / FMs)
- **Multi-task heads:** p_react, p_comment, p_share, p_dwell, p_negative — combined as a weighted utility
- **Training:** daily on impression+engagement logs; ~PB scale; A/B-tested before promotion
- **Serving:** accelerator pool (GPU/TPU); ~60 µs per candidate, 5K candidates per request → ~300 ms total

> **TIP**
>
> **Multi-task Utility**
>
> Don't just predict one engagement type. The serving formula combines several predicted probabilities with business-tuned weights:
> `U = a·p_react + b·p_comment + c·p_share + d·p_dwell − e·p_negative − f·p_clickbait`
> Tuning *a..f* is a product decision: heavy comment weight encourages discussion; heavy share weight pushes virality; the negative term protects long-term retention.

Negative Signals — Why They Matter

- **Hide:** demote anything similar (same author, same hashtag) for 30 days
- **Snooze 30d:** hard exclude — never rank from this source for the window
- **Unfollow:** remove from candidate generation entirely
- **Report:** route to integrity pipeline; possibly block author
- **Low-dwell:** implicit signal — < 1 s in viewport = mild demotion
- **Why critical:** negative signals are scarce but high-value; without them the ranker overfits to engagement bait

07 Storage — TAO, BLOB, Memcached

Polyglot persistence at planet scale

No single store fits feed. Posts and edges (friend, follow, like) live in **TAO** — a graph cache layered over sharded MySQL. Photos and videos live in the **BLOB store** (Haystack for warm, f4 for cold). Hot timeline lists, post metadata, and ranking features are kept in **Memcached**. Async work flows on **Kafka/Scribe**.

TAO — Graph Cache

- **Objects (nodes):** users, posts, comments, pages, groups
- **Associations (edges):** friend, follow, react, comment_on, share, member_of
- **Layout:** two-level cache (regional follower + per-shard leader) over sharded MySQL
- **Why graph?** Feed candidate gen is fundamentally *traverse the social graph*
- **Read path:** nearest cache → leader cache → MySQL primary; ~99% hit rate at the edge
- **Write path:** write to MySQL primary, invalidate cache, async replicate cross-region

Post Schema (TAO object)

```
# Object record
post {
  id:           bigint (snowflake)
  author_id:    bigint
  type:         enum {text, photo, video, link, life_event}
  text:         varchar(63206)        # FB max post length
  blob_refs:    list<blob_id>         # photos / video manifests
  audience:     enum {public, friends, friends_of_friends, custom}
  custom_acl:   list<user_id|list_id> # nullable
  created_at:   bigint (ms)
  edited_at:    bigint (ms) | null
  geo:          (lat, lng) | null
  lang:         varchar(8)
  integrity:    bitmask (clickbait, lowqual, fake_news, etc.)
  counters_id:  bigint                 # external counter row
}

# Association tables (sharded by 1st column)
friend(user_id, friend_id, created_at)
follow(user_id, page_id,   created_at)
member(user_id, group_id,  created_at, role)
react(post_id,  user_id,   reaction_type, created_at)
```

```
user_timeline(user_id, created_at DESC, post_id)   # push fan-out output
author_postlist(author_id, created_at DESC, post_id)  # pull-path source
```

BLOB Store — Haystack + f4

- **Haystack (warm):** append-only log files; needle index in RAM; one disk seek per photo
- **f4 (cold):** Reed-Solomon erasure-coded; ~2.1× storage overhead vs 3× replication
- **Tiering:** photos start in Haystack; if access rate < threshold for 90 days, migrate to f4
- **Upload path:** Post Service issues presigned URL; client PUTs bytes directly
- **Serving:** CDN absorbs ~95% of egress; origin pulls only on edge miss

Memcached — Hot Caches

Key	Contents	TTL	Why
tl:{user_id}	List of (post_id, ts, author_id) — push timeline	24 h sliding	Read on every feed open
post:{post_id}	Hydrated post (author name, counters, blob_refs)	1 h	Avoid TAO trip on hydration
counters:{post_id}	Reactions/comments/shares (sharded counters)	30 s	Hot updates; freshness matters
feat:user:{user_id}	Dense user features for ranker	5 min	Reused across consecutive feed reads
feat:post:{post_id}	Dense post features	1 h	Mostly static after the first hour
ranked:{user_id}:{cursor}	Stored top-K page	60 s	Idempotent re-pulls during scroll

INSIGHT

Lease Semantics For Counters

Memcached has no transactions. To prevent thundering-herd cache stampedes, Facebook added **leases**: the first cache miss is given a lease token; subsequent misses block briefly. The token-holder fetches from TAO and writes back; everyone else reads the freshly-cached value. Saves the DB from 1000× duplicate fills when a viral post explodes.

08 Write Path — Post Creation

From compose to fan-out

Step-by-Step

1. Client composes; if media, requests presigned BLOB URL via `POST /upload`
2. Client PUTs media bytes **directly** to BLOB store (Haystack)
3. Client calls `POST /post` with text, blob_refs, audience
4. Post Service validates: text length, audience ACL, integrity classifiers (NSFW, hate, misinformation)
5. TAO write: insert post object; insert edges (audience_acl, blob_refs, hashtags, mentions)
6. Append to `author_postlist(author_id)` — durable; supports pull-path reads
7. Emit `post_created` on Kafka; return 201 with post_id (no need to wait for fan-out)
8. Self-injection: client cache adds the post to the user's own feed view immediately (read-your-writes)
9. Fan-Out Worker (async): pushes to followers per §05 hybrid rules

> **TIP**
>
> **Why Async Fan-Out?**
>
> If we waited for fan-out before responding, a post by a user with 350 friends would block the API call for ~30 ms. Worse, a page with 100M followers would block for minutes. Async lets the write return in < 100 ms, and the worker absorbs the spike at its own pace. Followers see the post within 60 s — perceptually instant.

Integrity Pipeline (synchronous)

- **Hate / harassment:** classifier ensemble; block + appeal queue
- **Misinformation:** 3rd-party fact-check; demote not block
- **NSFW:** nudity classifier on media; age-gate or block
- **Spam:** velocity rules + ML; rate-limit account
- **Latency budget:** 50 ms p99 inline; deeper checks run async post-publish

09 Read Path — Feed Generation

Aggregate, rank, hydrate

Step-by-Step

1. Client calls `GET /feed?cursor=...`
2. Aggregator: ranked-feed cache check (key `ranked:{u}:{cursor}`); hit → return; miss → continue
3. Candidate gen: read `tl:{user_id}` from Memcached (push output)
4. Celeb pull: read `celeb_follows:{u}`; for each, fetch last N from `author_postlist`
5. Merge + dedup; cap at ~5,000 candidates by created_at
6. Filter: blocked, snoozed, hidden_similar, audience-mismatch, integrity-violations
7. Hydrate: feature vectors via `feat:user:{u}`, `feat:post:{p}`
8. Score: send ~5K candidates to Ranking Service → Model Server (DLRM)
9. Re-rank with diversity caps (≤2 per author per top-30); apply ad insertion slots
10. Hydrate metadata for top-K; store in `ranked:{u}:{cursor}` with 60 s TTL
11. Return top-30 + next_cursor; client renders; impressions logged async

Latency Budget (target p99 = 1.5 s end-to-end)

Stage	Budget	Notes
Network (client → edge)	150 ms	Geo-aware Anycast; TLS resume
Auth + routing	20 ms	Cached JWT verify
Aggregator candidate gen	100 ms	Memcached + TAO parallel calls
Filter + hydrate features	80 ms	Batched cache mget
Ranking inference (5K × 60 µs)	300 ms	Accelerator pool; batched
Re-rank + diversity + ads	30 ms	CPU
Hydrate top-K	70 ms	Parallel TAO/Memcached
Network (edge → client)	150 ms	Compressed JSON
Slack / GC / retries	300 ms	Headroom
Total p99	**1.20 s**	Inside 1.5s budget

WARNING

Pagination Stability

Cursors must be **stable** — the user must not see the same post twice on page 2 even though new posts arrived between page loads. The cursor encodes the ranked-feed snapshot ID + offset. Server stores the snapshot for 5 minutes; subsequent `GET /feed?cursor=...` reads from the same snapshot.

10 Action Logging & Training

Closing the ML loop

Ranking quality depends entirely on training data quality. Every impression, dwell, scroll, react, comment, share, hide, snooze must be logged with context — what was shown, what rank, what the user did. The volume is staggering: ~10B reads × 30 stories = **300B impressions/day**, before counting engagement events.

Event Schema

```
{
  "event_id":   "<uuid>",
  "viewer_id":  12345,
  "post_id":    67890,
  "author_id":  54321,
  "event_type": "impression | dwell | react | comment | share | hide | snooze | unfollow",
  "rank":       7,                        // position in feed
  "feed_session": "<uuid>",
  "model_id":   "dlrm_v143_2026_05_01",  // for offline replay
  "features":   { ... },                  // hashed feature vector
  "dwell_ms":   2400,                     // for impression events
  "timestamp":  "2026-05-07T10:30:45.123Z",
  "client":     { "app": "ios", "version": "442.1" }
}
```

Pipeline

- Client batches events; flushes every 5 s or on backgrounding
- API edge writes to Scribe / Kafka topic `feed_actions` (RF=3)
- Stream processor (Flink) does sessionisation + joins with the served snapshot
- Hourly: dump to **Hive / data warehouse** for offline training
- Daily: train next DLRM checkpoint; A/B test on 1% before global rollout
- Real-time: short-loop feedback (e.g. snooze, hide) updates user-feature cache within 60 s

> **INSIGHT**
>
> **Negative Sampling**
>
> Most impressions are non-engagements. To train, we don't need every non-event — we down-sample negatives ~10:1 against positives, then re-weight during training. Saves 90% of training compute with negligible AUC loss.

11 Caching Strategy

Layered, lease-protected, invalidation-aware

Cache Layers

Layer	What	TTL	Hit Rate (target)
CDN edge	Photo / video bytes	7 days	≥ 95%
Memcached (regional)	Push timeline tl:{u}	24 h slide	~98%
Memcached (regional)	Hydrated post post:{p}	1 h	~95%
Memcached (regional)	Counters counters:{p}	30 s	~85%
Memcached (regional)	User / post features	5 min / 1 h	~92%
Memcached (regional)	Ranked-feed snapshot	60 s	~70% (during scroll)
TAO leader cache	Posts, edges	until invalidate	~99%
Client SDK	Pre-rendered top-30 + next 30	session	100% on scroll

Invalidation

- **Write-through invalidation:** writer publishes invalidate event; cache nodes drop the key
- **Lease tokens:** first miss gets a lease; others wait briefly to avoid stampede
- **TTL backstop:** if an invalidate is dropped, key expires anyway
- **Counter sloppiness:** reaction counts shown ±1% acceptable; serves from coarse-grained sharded counters

> **TIP**
>
> **Two-Level Cache**
>
> TAO uses a **region-local follower cache** in front of a **per-shard leader cache** in front of MySQL. A miss at the follower walks up to the leader, then to MySQL. Writes go to MySQL primary and invalidate the leader (and via gossip, the followers). Net effect: > 99% hit rate at the regional edge, low cross-DC traffic.

12 Comparable Systems

News Feed vs Twitter timeline vs Instagram

All three are timelines built on hybrid push/pull, but they diverge sharply in graph shape, content type, and ranking objective.

Aspect	Facebook News Feed	Twitter Home Timeline	Instagram Feed
Graph	Bidirectional friends + follows of pages/groups	Asymmetric follow graph; many-to-many	Asymmetric follow; typically ≪ friends
Avg sources	~550 (350 friends + pages/ groups)	~400 follows (heavy-tailed)	~150 follows
Content	Mixed: text/photo/video/link/ life events/ads	Mostly short text + media	Photo + video first
Ranking	DLRM, multi-task, integrity-aware	Earlybird → real-time graph + Heavy Ranker	Two-tower; engagement+recency
Freshness target	< 60 s for friends	Near real-time (seconds)	< 60 s
Push threshold	~100K followers	~1M followers (lower w_push)	~1M followers
Storage primary	TAO (graph cache + MySQL)	Manhattan (KV) + Gizzard / Twemcache	MySQL + Cassandra
Media store	Haystack + f4	Blobstore	S3
Distinct twist	Pages + groups inflate fanout vs Twitter	Retweet adds extra fan-out edge	Stories run on a separate stack

INSIGHT

Why The Threshold Differs

Twitter's push threshold is higher (~1M) because their write-cost-per-follower is lower (smaller payload, simpler timeline rows) and their read freshness demand is higher (real-time culture). Facebook's News Feed tolerates 60 s lag in exchange for richer hydrated metadata at push time. Different constants, same hybrid pattern.

13 Scalability & Sharding

Horizontal everything

Sharding Plan

Data	Shard Key	Shards	Why
users	hash(user_id) % 4096	4096+	Even distribution; isolate hot users
posts	hash(author_id) % 4096	4096+	Co-locate author's posts; profile reads stay single-shard
friend / follow edges	hash(user_id) % 4096	4096+	User's outgoing edges all on one shard
user_timeline (push)	hash(user_id) % 8192	8192+	Hottest write target during fan-out
author_postlist (pull)	hash(author_id) % 4096	4096+	Reads concentrated on celebs — pre-warm caches
counters	hash(post_id) % 16384	16384+	Hot reactions on viral posts; sub-shard counters per region
BLOB Haystack	consistent hash on blob_id	1000s of volumes	Append-only logs; rebalance via rolling migration

Replication & Multi-Region

- **Single-master writes per user:** user is pinned to a primary region (Atlanta, Luleå, Singapore...)
- **Cross-region replication:** async; ~1 s typical lag
- **Read locally:** all read paths satisfied from the nearest region
- **TAO follower cache:** per region; invalidations cross-region via the wormhole stream
- **Failover:** if a primary region drops, secondary promotes after coordination quorum

WARNING

Hot Author Mitigation

A page with 200M followers posting concurrently with 9 other pages causes **2B writes** in seconds if naïvely pushed. Mitigations: (1) celeb threshold skips push entirely; (2) sharded counters break per-post hotspots; (3) rate-limit rapid-fire pages to once per N seconds; (4) batch LPUSHes per follower-shard to one RPC.

14 Failure Modes & Recovery

Graceful degradation order

Failure	Impact	Detection	Mitigation
Memcached node down	Higher TAO load on miss	Health check + miss rate alarm	Consistent-hash drop; TAO absorbs; spin replacement; lease tokens limit stampede
TAO leader down	Reads slower, writes blocked on shard	Replica-lag + write-error alarm	Promote replica; reroute; backfill from MySQL primary
MySQL shard down	That shard's reads/ writes fail	Connection timeout	Auto-promote replica (Orchestrator); ~30 s RTO; cache absorbs
Ranking Model Server down	No personalised ranking	Inference latency / error rate	Fall back to chronological merge; serve cached ranked feed; banner 'updating'
Fan-Out Worker lag	Friend posts late to followers	Kafka consumer-lag metric	Scale workers; pull-path rescues during outage; freshness SLO breach
Kafka broker partition	Action logging stalls; training slips a day	Broker dead alert	RF=3 brokers; 7-day buffer; consume from the surviving region
BLOB store outage	New uploads fail; existing media OK from CDN	PUT error rate	Queue uploads in client; presigned URL retry; user banner
CDN regional outage	+200 ms latency in that region	CDN provider alert	DNS failover to backup CDN; origin scales
Cache stampede on viral post	TAO request floods	Per-key miss rate spike	Lease tokens; coalesced fills; pre-warm celeb posts
Cross-region partition	Read-your-writes broken cross-region	Latency + replication lag	Pin user to primary region; serve stale until heal
Cascading slowness	Timeouts propagate up	p99 explosion	Circuit breakers around Ranking; fall back to cache; shed traffic

WARNING

Degradation Order

> If we have to drop features to stay up, the order is: (1) ML ranking → chronological merge; (2) hydration richness → strip side-counters; (3) ad insertion → skip; (4) celeb pull → skip; (5) action logging → buffer locally; (6) **feed read of pre-built timeline is never dropped**.

Disaster Recovery

- **RTO:** < 30 s for a single shard; < 5 min for a regional outage
- **RPO:** < 1 s for posts (synchronous WAL); < 1 min for action logs
- **Backups:** MySQL incremental every 30 min; full daily; off-region copies
- **Drills:** quarterly chaos exercises take a region offline for one hour

15 Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Fan-out model	Hybrid push/pull at ~100K threshold	Push-only melts on big pages; pull-only melts on read scatter-gather. Hybrid pays more storage to serve fast reads for the 99% case and cheap writes for the heavy-tail.
Ordering	ML-ranked, not chronological	Better engagement and time-spent; opaque to users; risk of filter bubbles. Chronological is honest but burys quality content.
Consistency	Eventual for feed; read-your-writes for poster	Stale by < 60 s for friends; needs self-injection for the poster. Strong consistency at this scale is impossible across regions.
Storage	TAO over sharded MySQL; Haystack+f4	Operational complexity vs single store. TAO cache layer is a Facebook-grade investment; smaller orgs use Cassandra+S3.
Ranking model	DLRM with multi-task heads	PB of training data; 10s of TB of embeddings; serving cost is real. Linear EdgeRank is cheaper but plateaus on quality.
Cache TTL	Counters 30s, posts 1h, timelines 24h slide	Counters trade freshness for cluster load; long TTLs save fills but risk stale UX.
Self-injection (RYW)	Client-side overlay	Simple; requires careful dedup when push catches up. Server-side strict RYW would force read-after-write across replicas.
Negative signals	Strong demotion + 30-day exclusion	Protects retention but reduces candidate diversity; tunable per-account.
Push threshold	~100K (operational sweet spot)	Higher threshold saves writes but risks read-path scatter-gather; tune per cluster capacity.
Action logging	At-least-once, sampled negatives	Cheap and accurate enough for ML; exactly-once is unimplementable across systems.

INSIGHT

The Defining Trade-Off

News Feed exists at the intersection of **graph traversal** (social), **information retrieval** (relevance ranking), and **real-time messaging** (freshness). Every architectural choice is the cheapest answer to *two of those* while paying down the third in latency, storage, or staleness.

16 Interview Playbook

45-minute execution

News Feed is an everyone-asks-it design. The expected depth: hybrid fan-out math, ML ranking layer, integrity, scale numbers. Most candidates hand-wave the threshold; the strong ones derive it.

45-Minute Time Budget

Window	Phase	Goal
00:00–02:00	Clarify requirements	Friends + pages + groups; ML ranked; 2B DAU
02:00–07:00	Capacity estimation	1B posts/day, 10B reads/day, 115K avg / 300K peak QPS
07:00–17:00	High-level architecture	Aggregator + Ranking + Action Logger + TAO + BLOB + Memcached + Kafka
17:00–32:00	Hybrid fan-out deep dive	Push vs pull, threshold math, worker pseudocode
32:00–40:00	Ranking stack	EdgeRank → DLRM, signals, multi-task, negative signals, action logging
40:00–45:00	Trade-offs + failures	Degradation order, sharding, hot pages

Key Talking Points

- **“2B DAU × 5 feeds / 86,400 = 115K reads/sec; peak ~300K”** — capacity math, recited
- **“1B posts/day → 12K writes/sec; engagement ~10× → 120K/sec”**
- **“Hybrid push/pull; threshold ~100K followers”**
- **“Threshold isn't a magic number — it's where burst push exceeds cluster capacity”**
- **“Self-injection delivers read-your-writes without strict consistency”**
- **“5K candidates × 60 µs per score = 300 ms ranking budget”**
- **“DLRM with multi-task heads (react, comment, share, dwell) − negative signals”**
- **“Action logging closes the loop — 300B impressions/day fuel daily training”**
- **“TAO over MySQL + Haystack/f4 + Memcached — polyglot at planet scale”**
- **“Degrade ranking before dropping the feed itself”**

Common Follow-ups & Strong Answers

- **Q: Why hybrid, not pure pull?** A: Pure pull does 550-source scatter-gather per read at 115K reads/sec. The aggregate read load on TAO postlists would be ~63M partition reads/sec — far above cluster capacity. Push pre-builds 99% of the work.
- **Q: Why not pure push?** A: A page with 100M followers posting causes 100M LPUSH in seconds. We'd need an order-of-magnitude bigger Memcached cluster just to absorb the burst. Pull amortises celeb cost across the day.
- **Q: How do you pick the threshold?** A: It's the F where $F \cdot w_{push}$ exceeds your cluster's burst capacity per author class. We measure: at ~100K followers, a single post saturates one Memcached pod for ~2 s; that's our operational ceiling.
- **Q: How does a poster see their own post instantly?** A: Self-injection — the client overlays the freshly-created post on top of the timeline locally. The async fan-out catches up within 60 s; dedup by post_id on overlap.
- **Q: What if the ranking model is down?** A: Aggregator falls back to chronological merge of candidates and serves cached top-K from before the outage; banner indicates 'feed updating'.
- **Q: How do negative signals work?** A: Hide → 30-day demotion of similar (same author, hashtag); snooze → hard exclusion from candidate gen; unfollow → graph edit. They're scarce but high-value training signals.
- **Q: How do you avoid filter bubbles?** A: Diversity caps in re-rank (max 2 stories per author in top-30); injecting exploration items at low rate; integrity classifiers demote echo-chamber bait.
- **Q: Counter consistency?** A: Sloppy — we use sub-shard counters and accept ±1% on viral posts. Exactness costs latency and almost no user notices.

KEY

Numbers to Memorise

3B MAU / 2B DAU · **1B posts/day** (12K/sec) · **10B feed reads/day** (115K avg, 300K peak) · **5K candidates / 60 µs each / 300 ms total** · **~30 stories per page** · **~100K push/pull threshold** · **~6 TB hot Memcached** · **300B impressions/day** for training.

TIP

If You Get Stuck

Anchor on the cost model. State the variables: F (followers), R (reads per follower per day), w_push, w_pull. Write the inequality. The interviewer wants to see you reason about *why* the threshold exists, not memorise that it does.